Part IV

Gradient Methods

Lecture Outline

1. Steepest (or gradient) descent
2. Newton (Raphson) method
3. Gauss-Newton method
4. Stochastic gradient descent

Introduction

In this lecture we will discuss algorithms for solving uncontrained optimization problems:

$$\min_x f(x)$$

The optimization algorithms that we will discuss can be grouped into three classes

- Zero order methods
- First order methods
- Second order methods

First order methods requires that the cost function f is C^1 and use the gradient, second order methods requires that f is C^2 and use the gradient and Hessian. First and second order methods are collectively called gradient methods. Zero order methods do not require any differentiability assumptions on f.

Steepest (or gradient) descent method

Recall that the gradient $\nabla f(x)$ (of f at x) points in the direction of steepest-ascent from x, hence

$-\nabla f(x)$ points in the direction of steepest descent from x

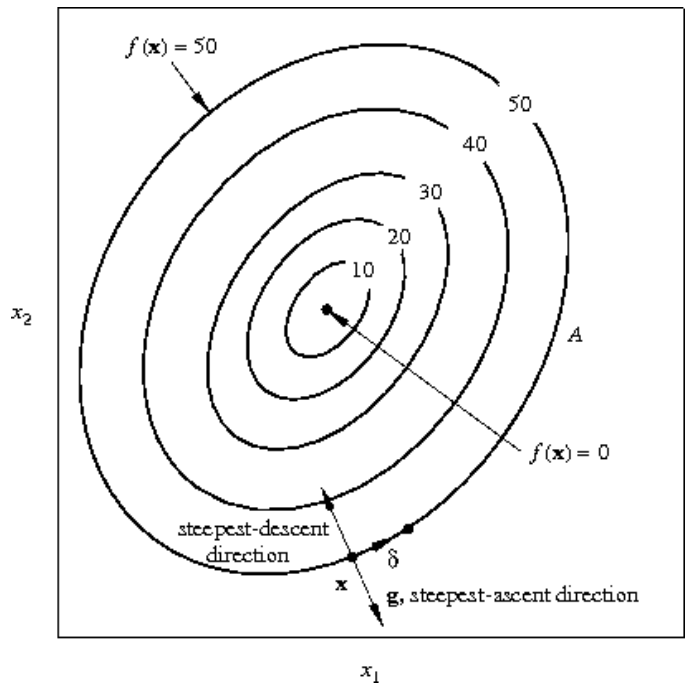


Figure: $\mathbf{g} = \nabla f(\mathbf{x})$

Steepest (or gradient) descent method

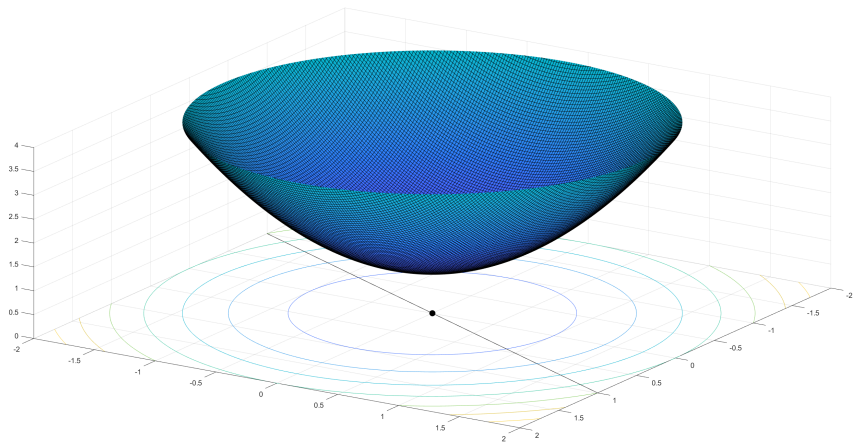
From the observations regarding the gradient we conclude that

$$f(x) > f(x + \alpha d), \qquad d = -\nabla f(x)$$

for some $\alpha \geq 0$ if x is not an extremum point.

Hence if $d = -\nabla f(x)$ points in the direction of a minimizer (for all x), then the optimization problem can be formulated as a one dimensional optimization

$$\min_{\alpha \geq 0} f(x + \alpha d)$$



In general this does (of course) not work.

Steepest (or gradient) descent method

A general iterative approach to finding a minimizer can be described as follows:

Choose a start point x_0 and set $d_0 = -\nabla f(x_0)$. Let α_0 be the solution to

$$\min_{\alpha \geq 0} f(x_0 + \alpha d_0)$$

Set $x_1 = x_0 + \alpha_0 d_0$ and $d_1 = -\nabla f(x_1)$. Let α_1 be the solution to

$$\min_{\alpha \geq 0} f(x_1 + \alpha d_1)$$

etc.

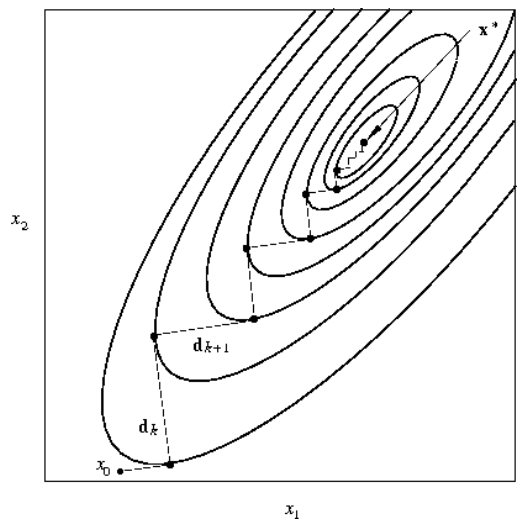


Figure: Solution trajectory for steepest decent method.

Steepest (or gradient) descent method

Note that the method above assumes C^1 information and that it may be described compactly as

$$x_{k+1} = x_k + \alpha_k d_k, \ \ k = 0, 1, 2, \ldots$$

with

$$\begin{aligned} &x_0 \text{ a given start point,} \\ &\alpha_k \in \operatorname{argmin}_{\alpha \geq 0} f(x_k + \alpha d_k), \\ &d_k = -\nabla f(x_k). \end{aligned}$$

Moreover, successive directions d_k and d_{k+1} are orthogonal:

$$0 = \frac{d}{d\alpha} f(x_k + \alpha_k d_k) = \nabla f(x_k + \alpha_k d_k)^T d_k = -d_{k+1}^T d_k.$$

How to implement this method?

Steepest (or gradient) descent algorithm

Step 1: • Input: cost function f and its gradient ∇f, initial point x_0, tolerance ϵ, and maximum number of iterations max_iter

Step 2: • For $k = 0, 1, 2, \ldots,$ max_iter

Step 2.1: Calculate the gradient $\nabla f(x_k)$ and set $d_k = -\nabla f(x_k)$

Step 2.2: Find α_k as $\alpha_k \in \operatorname{argmin}_{\alpha \geq 0} f(x_k + \alpha d_k)$

Step 2.3: Set $x_{k+1} = x_k + \alpha_k d_k$

Step 2.4: If $\|\alpha_k d_k\| < \epsilon$ or $k =$ max_iter

Output: $x' = x_{k+1}$, $f(x')$ and break

There are several highly efficient methods for solving the (one dimensional) optimization problem in Step 2.2. Below we discuss two such methods:

- Golden section search
- Backtracking line search

both being (inexact) line search methods.

Golden section search algorithm

Step 1:

- Input: cost function f, lower interval bounds a, upper interval bounds b, tolerance ϵ and maximum number of iterations max_iter (optional)

Step 2

- Initialization:

$$\phi = \frac{1+\sqrt{5}}{2} \approx 1.618 \text{ (the golden ratio)}, \quad \psi = 2 - \phi \approx 0.382$$
$$x_1 = a + \psi(b-a), \quad x_2 = b - \psi(b-a)$$
$$f_1 = f(x_1), \quad f_2 = f(x_2)$$

Step 3:

- While $|b-a| > \epsilon$ and iter $<$ max_iter
- If $f_1 < f_2$ then

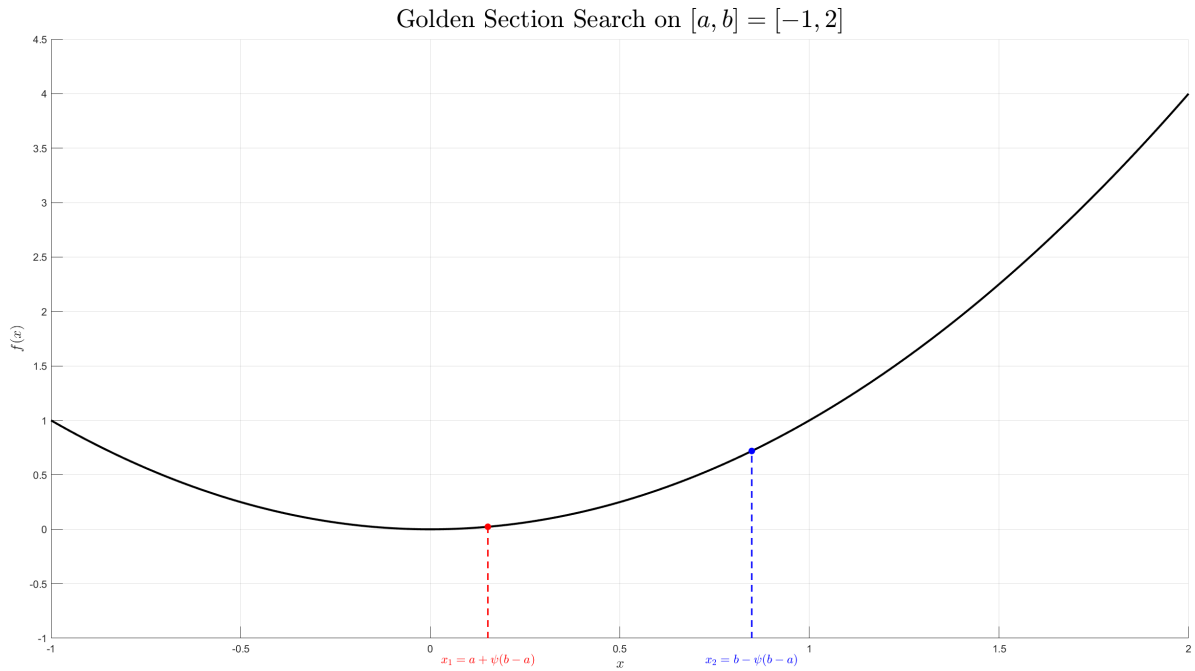
$$b = x_2,\ x_2 = x_1,\ f_2 = f_1,\ x_1 = a + +\psi(b-a),\ f_1 = f(x_1),\ \text{iter} = \text{iter} + 1$$

- Else

$$a = x_1,\ x_1 = x_2,\ f_1 = f_2,\ x_2 = b - \psi(b-a),\ f_2 = f(x_2),\ \text{iter} = \text{iter} + 1$$

Step 4:

- Output: $x_{\text{min}} = \frac{a+b}{2}$ and $f(x_{\text{min}})$



Golden section search algorithm

Step 1:
- Input: cost function f, lower interval bounds a, upper interval bounds b, tolerance ϵ and maximum number of iterations max_iter (optional)

Step 2
- Initialization:

$$\phi = \frac{1+\sqrt{5}}{2} \approx 1.618 \text{ (the golden ratio)}, \quad \psi = 2 - \phi \approx 0.382$$
$$x_1 = a + \psi(b-a), \quad x_2 = b - \psi(b-a)$$
$$f_1 = f(x_1), \quad f_2 = f(x_2)$$

Step 3:
- While $|b-a| > \epsilon$ and iter $<$ max_iter
- If $f_1 < f_2$ then

$$b = x_2,\ x_2 = x_1,\ f_2 = f_1,\ x_1 = a + +\psi(b-a),\ f_1 = f(x_1),\ \text{iter} = \text{iter} + 1$$

- Else

$$a = x_1,\ x_1 = x_2,\ f_1 = f_2,\ x_2 = b - \psi(b-a),\ f_2 = f(x_2),\ \text{iter} = \text{iter} + 1$$

Step 4:
- Output: $x_{\min} = \frac{a+b}{2}$ and $f(x_{\min})$

The golden section search algorithm is designed for one dimensional optimization problems and can be applied to Step 2.2 of the steepest descent algorithm

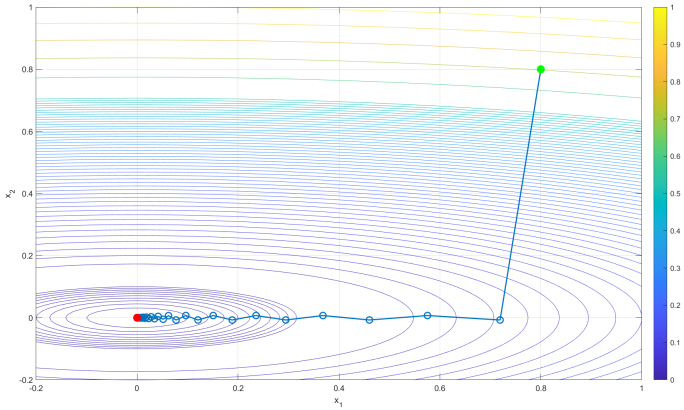
$$\text{Find } \alpha_k \text{ as } \alpha_k \in \text{argmax}_{\alpha \geq 0} f(x_k + \alpha_k d_k)$$

by choosing $a = x_k$, $b = x_k + \hat{\alpha} d_k$ with $\hat{\alpha}$ a fixed tuning parameter, $f(x) = f(x_k + x d_k)$, and replacing $|a-b|$ with $\|a-b\|_2$. Step 2.2 is then solved by

$$\alpha_k = \|x_{\min} - x_k\|_2 / \|d_k\|_2$$

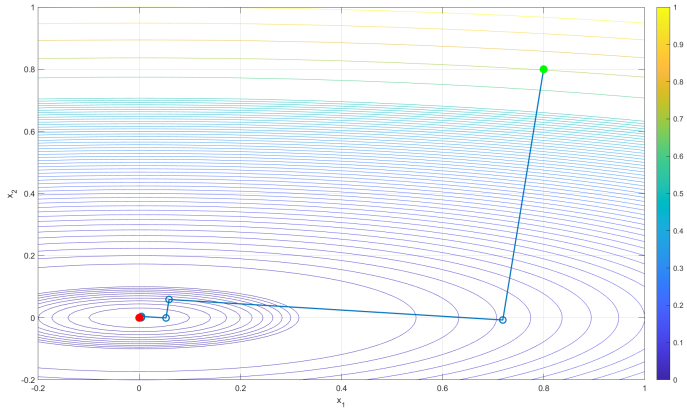
Steepest descent with golden section search

Cost function $f(x) = 0.1x_1^2 + x_2^2$



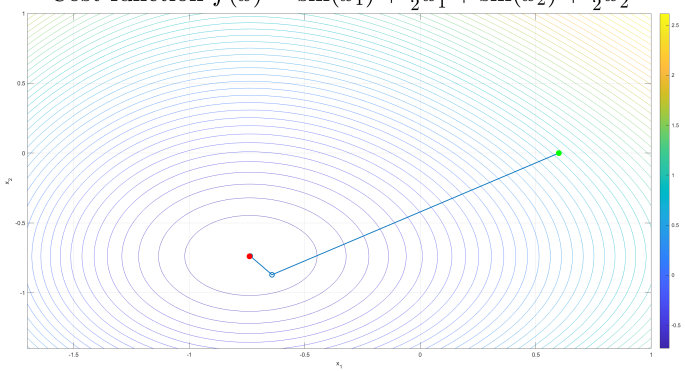
(a) $\hat{\alpha} = 1$ and end at $k = 59 <$ max_iter=100

Cost function $f(x) = 0.1x_1^2 + x_2^2$



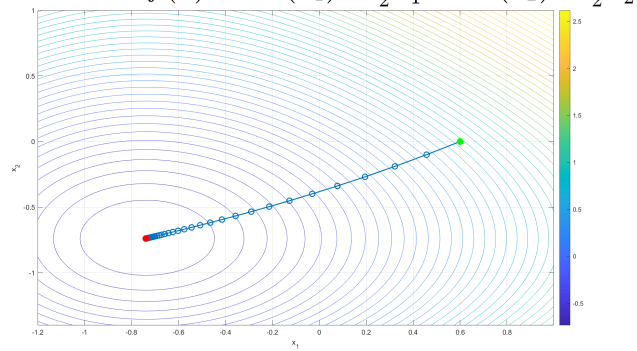
(b) $\hat{\alpha} = 5$ and end at $k = 11 <$ max_iter=100

Cost function $f(x) = \sin(x_1) + \frac{1}{2}x_1^2 + \sin(x_2) + \frac{1}{2}x_2^2$



(c) $\hat{\alpha} = 1$ and end at $k = 4 <$ max_iter=100

Cost function $f(x) = \sin(x_1) + \frac{1}{2}x_1^2 + \sin(x_2) + \frac{1}{2}x_2^2$



(d) $\hat{\alpha} = 0.1$ and end at $k = 72 <$ max_iter=100

Backtracking line search algorithm

Step 1:

- Input: cost function f and its gradient ∇f, initial point x, maximum number of iterations max_iter, Armijo constant $\hat{\alpha} \in (0, 0.5)$, step size reduction factor $\beta \in (0, 1)$, and search direction $d \in \mathbb{R}^n$

Step 2

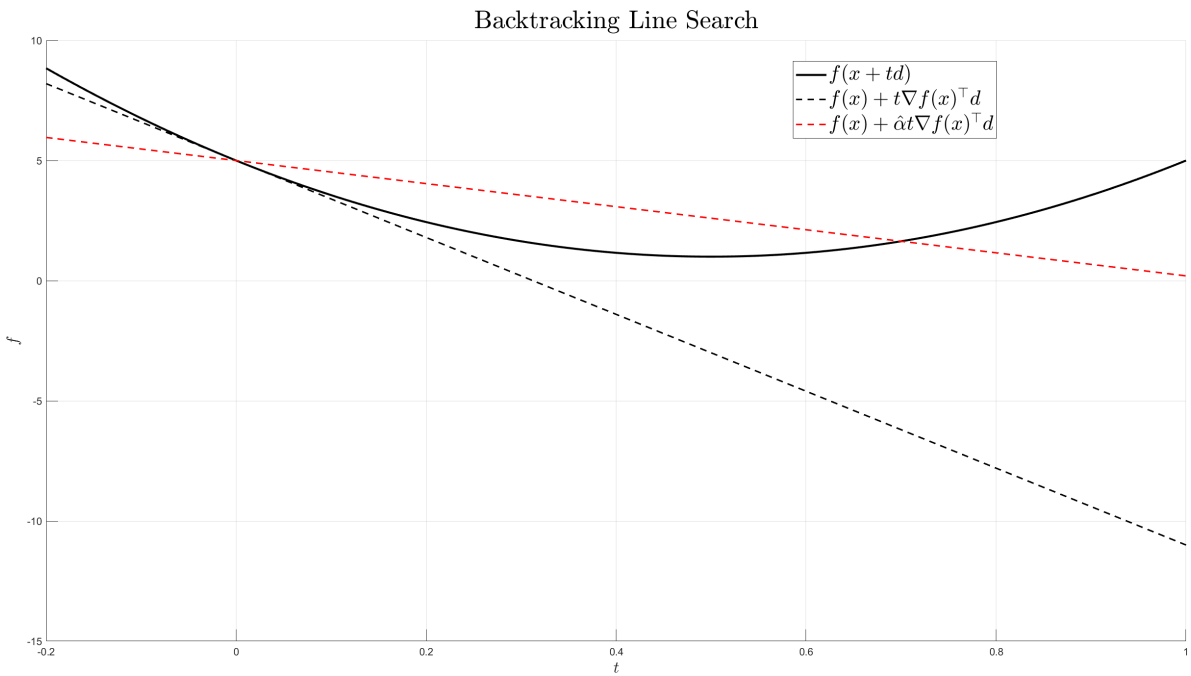
- Initialization:
  $t = 1$ (initial step size)
  iter $= 0$ (iteration count)

Step 3:

- While $f(x + td) > f(x) + \hat{\alpha} t \nabla f(x)^\top d$ and iter $<$ max_iter
  $t = \beta t$
  iter $=$ iter $+ 1$

Step 4:

- Output: t



Backtracking line search algorithm

Step 1:
- Input: cost function f and its gradient ∇f, initial point x, maximum number of iterations max_iter, Armijo constant $\hat{\alpha} \in (0, 0.5)$, step size reduction factor $\beta \in (0, 1)$, and search direction $d \in \mathbb{R}^n$

Step 2
- Initialization:
  $t = 1$ (initial step size)
  iter $= 0$ (iteration count)

Step 3:
- While $f(x + td) > f(x) + \hat{\alpha} t \nabla f(x)^\top d$ and iter $<$ max_iter
  $t = \beta t$
  iter $=$ iter $+ 1$

Step 4:
- Output: t

The backtracking line search algorithm can be applied to Step 2.2 of the steepest descent algorithm

$$\text{Find } \alpha_k \text{ as } \alpha_k \in \operatorname{argmin}_{\alpha \geq 0} f(x_k + \alpha_k d_k)$$

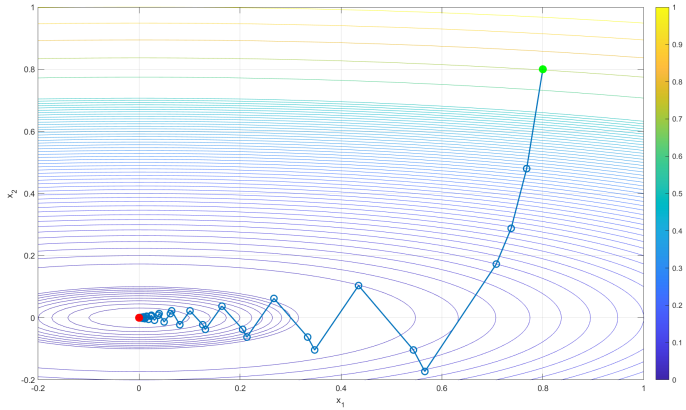
by choosing $x = x_k$ and $d = d_k$, which produce the solution

$$\alpha_k = t$$

The constants $\hat{\alpha}$ and β are typically chosen in $(0.01, 0.3)$ and $(0.1, 0.8)$, respectively.

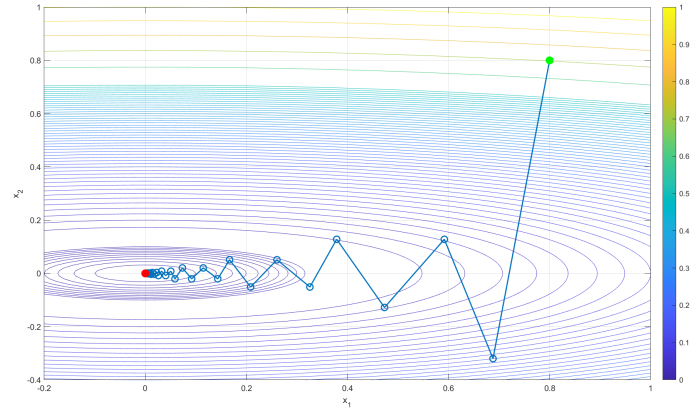
Steepest descent with backtracking line search

Cost function $f(x) = 0.1x_1^2 + x_2^2$



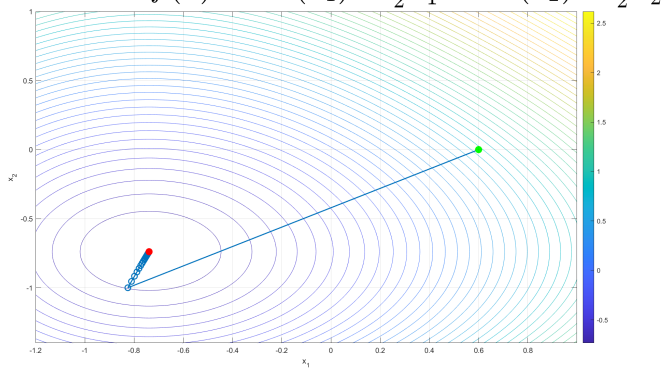
(e) $\hat{\alpha} = 0.1$, $\beta = 0.2$ and end at $k = 73 <$ max_iter=100

Cost function $f(x) = 0.1x_1^2 + x_2^2$



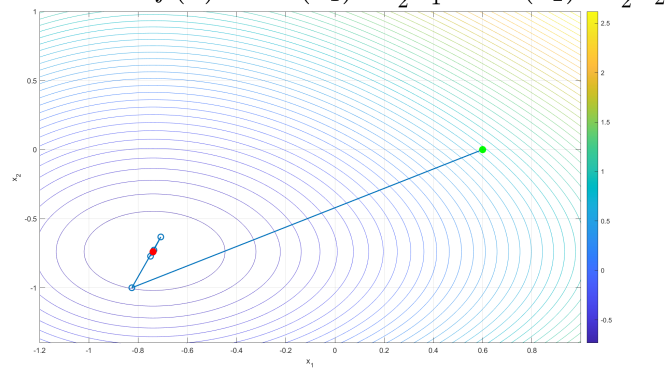
(f) $\hat{\alpha} = 0.1$. $\beta = 0.7$ and end at $k = 63 <$ max_iter=100

Cost function $f(x) = \sin(x_1) + \frac{1}{2}x_1^2 + \sin(x_2) + \frac{1}{2}x_2^2$



(g) $\hat{\alpha} = 0.2$, $\beta = 0.1$ and end at $k = 61 <$ max_iter=100

Cost function $f(x) = \sin(x_1) + \frac{1}{2}x_1^2 + \sin(x_2) + \frac{1}{2}x_2^2$



(h) $\hat{\alpha} = 0.2$, $\beta = 0.8$ and end at $k = 14 <$ max_iter=100

Steepest (or gradient) descent method

There are also ways to avoid determining α_k by the line search in Step 2.2 of the steepest descent algorithm. Without going into details here are three such possibilities:

1. $\alpha_k \approx \frac{-\nabla f(x_k)^T d_k}{d_k^T H_f(x_k) d_k} = \frac{\|\nabla f(x_k)\|_2^2}{\nabla f(x_k)^T H_f(x_k) \nabla f(x_k)} = \frac{\|d_k\|_2^2}{d_k^T H_f(x_k) d_k}$
2. $\alpha_k \approx \frac{\hat{\alpha}^2 ||d_k||_2^2}{2(f(x_k+\hat{\alpha} d_k) - f(x_k) + \hat{\alpha}||d_k||_2^2)}$, with $\hat{\alpha} = \alpha_{k-1}$, $k > 1$ (the previous optimum) and $\hat{\alpha} = 1$ for $k = 1$.
3. $\alpha_k = \hat{\alpha}$, **with $\hat{\alpha}$ a fixed tuning constant, sometimes called a learning rate.**

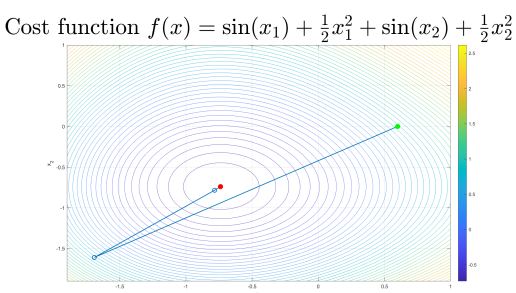
Technical remark: Item 1 and 2 are obtained via quadratic approximation of f

$$f(x_k + \alpha d_k) \approx f(x_k) - \alpha ||d_k||_2^2 + \frac{1}{2}\alpha^2 d_k^T H_f(x_k) d_k$$

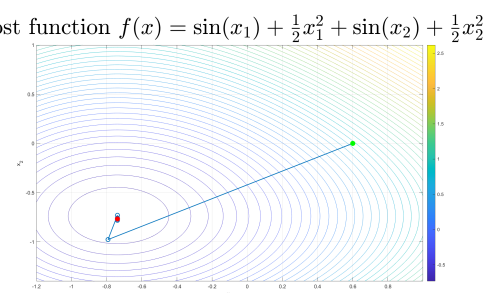
Steepest (or gradient) descent method

There are also ways to avoid determining α_k by the line search in Step 2.2 of the steepest descent algorithm. Without going into details here are three such possibilities:

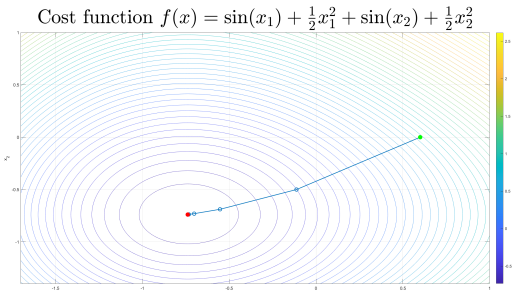
1. $\alpha_k \approx \dfrac{-\nabla f(x_k)^T d_k}{d_k^T H_f(x_k) d_k} = \dfrac{\|\nabla f(x_k)\|_2^2}{\nabla f(x_k)^T H_f(x_k) \nabla f(x_k)} = \dfrac{\|d_k\|_2^2}{d_k^T H_f(x_k) d_k}$
2. $\alpha_k \approx \dfrac{\hat{\alpha}^2 ||d_k||_2^2}{2(f(x_k+\hat{\alpha} d_k) - f(x_k) + \hat{\alpha}||d_k||_2^2)}$, with $\hat{\alpha} = \alpha_{k-1}$, $k > 1$ (the previous optimum) and $\hat{\alpha} = 1$ for $k = 1$.
3. $\alpha_k = \hat{\alpha}$, **with $\hat{\alpha}$ a fixed tuning constant, sometimes called a learning rate.**



(i) Using (1): End at $k = 5 <$ max_iter=100



(j) Using (2): End at $k = 19 <$ max_iter=100

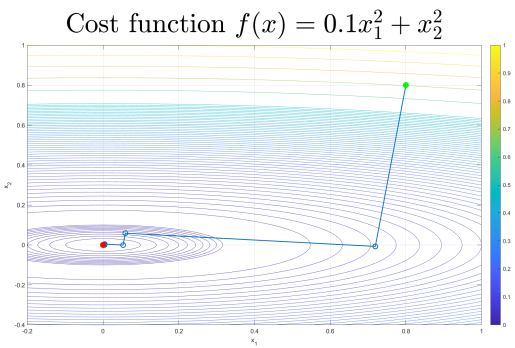


(k) Using (3): $\hat{\alpha} = 0.5$ and end at $k = 10 <$ max_iter=100

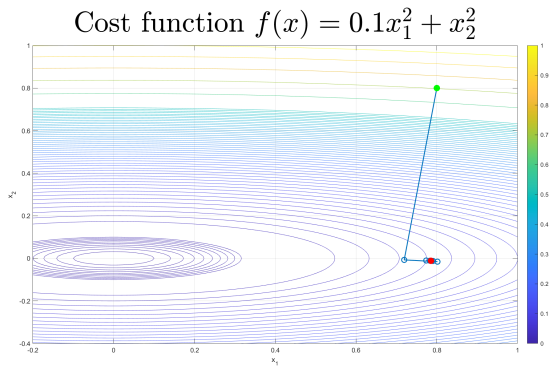
Steepest (or gradient) descent method

There are also ways to avoid determining α_k by the line search in Step 2.2 of the steepest descent algorithm. Without going into details here are three such possibilities:

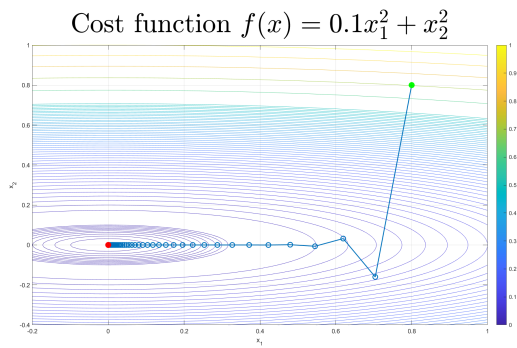
1. $\alpha_k \approx \dfrac{-\nabla f(x_k)^T d_k}{d_k^T H_f(x_k) d_k} = \dfrac{\|\nabla f(x_k)\|_2^2}{\nabla f(x_k)^T H_f(x_k) \nabla f(x_k)} = \dfrac{\|d_k\|_2^2}{d_k^T H_f(x_k) d_k}$
2. $\alpha_k \approx \dfrac{\hat{\alpha}^2 ||d_k||_2^2}{2(f(x_k+\hat{\alpha} d_k) - f(x_k) + \hat{\alpha}||d_k||_2^2)}$, with $\hat{\alpha} = \alpha_{k-1}$, $k > 1$ (the previous optimum) and $\hat{\alpha} = 1$ for $k = 1$.
3. $\alpha_k = \hat{\alpha}$, **with $\hat{\alpha}$ a fixed tuning constant, sometimes called a learning rate.**



(l) Using (1): End at $k = 13 <$ max_iter=100



(m) Using (2): End at $k = 18 <$ max_iter=100 **FAILED**



(n) Using (3): $\hat{\alpha} = 0.6$ and end at $k = 91 <$ max_iter=100 (**FAIL** for $\hat{\alpha} = 1$ and reach max_ither for $\hat{\alpha} < 0.55$)

Technical remark: The set $\rho(H_f(x))$ of eigenvalues of the Hessian of f at x determines the convergence property of steepest descent algorithm

Theorem

Assume that $f \in C^2$ with $H_f(x^*) > 0$ for some minimizer x^* and let $\{x_k\}$ denote the sequence generated by the steepest-descent algorithm. Then for x_k sufficiently close to x^*

$$f(x_{k+1}) - f(x^*) \leq \left(\frac{1-r}{1+r}\right)^2 (f(x_k) - f(x^*)),$$

or equivalently

$$\frac{f(x_{k+1}) - f(x^*)}{f(x_k) - f(x^*)} \leq \left(\frac{1-r}{1+r}\right)^2,$$

with **condition number**

$$r = \frac{\min \rho(H_f(x_k))}{\max \rho(H_f(x_k))}.$$

Hence

- Fast convergence for $\min \rho(H_f(x_k)) \approx \max \rho(H_f(x_k))$ (implying $r \approx 1$).
- Slow convergence for $\min \rho(H_f(x_k)) \ll \max \rho(H_f(x_k))$ (implying $r \approx 0$).

Newton (Raphson) algorithm

Using $-\nabla f$ as a search direction is not always the optimal choice. Close to a minimizers it can be advantious to use the Hessian in combination with the gradient in the design of a search direction, as in the Newton (Raphson) algorithm:

Step 1:
- Input: cost function f its gradient ∇f and Hessian H_f, initial point x_0, tolerance ϵ, and maximum number of iterations max_iter

Step 2:
- For $k = 0, 1, 2, \ldots,$ max_iter

Step 2.1: Calculate the gradient $\nabla f(x_k)$ and the Hessian $H_f(x_k)$
Step 2.2: If $H_f(x_k)$ is not positive definite then make it positive definite.
Step 2.3: Calculate $H_f(x_k)^{-1}$ and set $d_k = -H_f(x_k)^{-1}\nabla f(x_k)$
Step 2.4: Find α_k as $\alpha_k \in \text{argmin}_{\alpha \geq 0} f(x_k + \alpha d_k)$
Step 2.5: Set $x_{k+1} = x_k + \alpha_k d_k$
Step 2.6: If $\|\alpha_k d_k\| < \epsilon$ or $k =$ max_iter
Output: $x' = x_{k+1}$, $f(x')$ and break

There are several ways to obtain a positive definite modification $H_f(x_k)'$ of the Hessian $H_f(x_k)$ as required by Step 2.2.

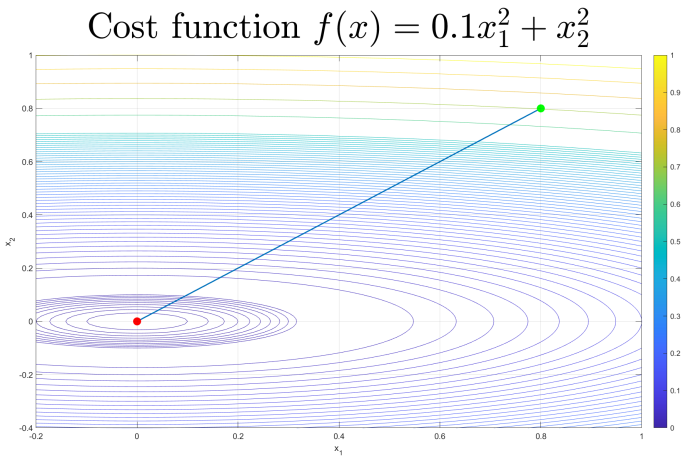
"Steepest-descent":

$$H_f(x_k)' = 1/(1+\beta)H_f(x_k) + \beta/(1+\beta)I,$$

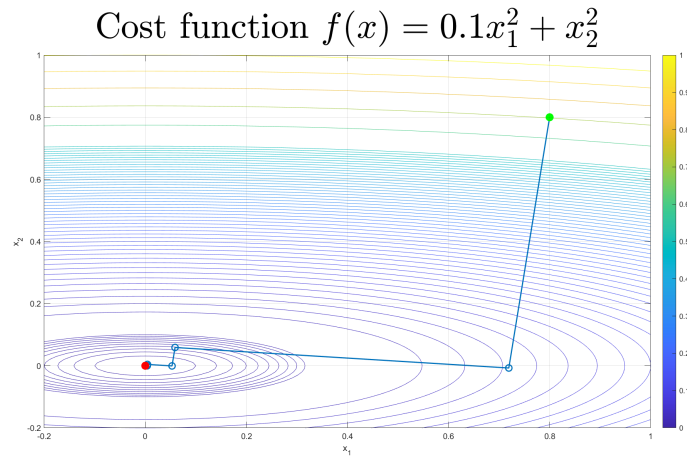
with β small (resp. large) if $H_f(x_k) > 0$ (resp. $H_f(x_k) \leq 0$).

Steepest descent vs Newton Raphson

Compareson between steepest descent and Newton Raphson when using golden section search



(o) $\hat{\alpha} = 5$ and end at $k = 4 <$ max_iter=100

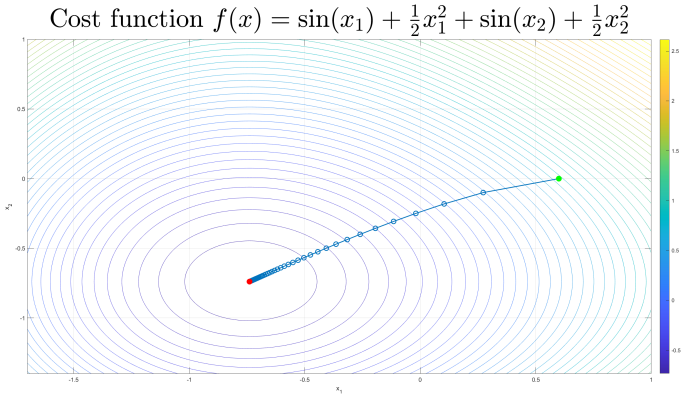


(p) $\hat{\alpha} = 5$ and end at $k = 11 <$ max_iter=100

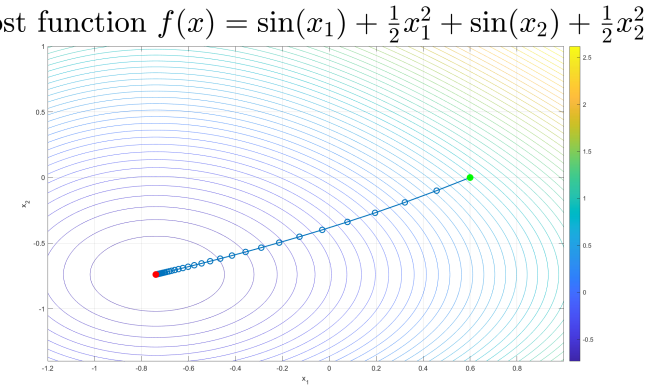
Figure: Left: Newton Raphson. Right: Steepest descent

Steepest descent vs Newton Raphson

Compareson between steepest descent and Newton Raphson when using golden section search



(a) $\hat{\alpha} = 0.1$ and end at $k =$ max_iter=100

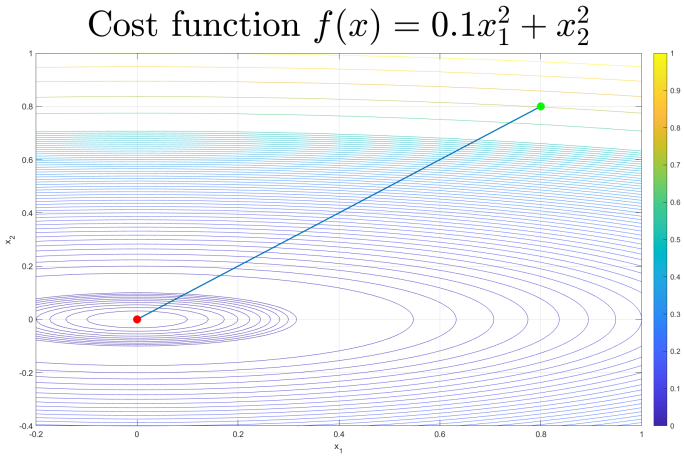


(b) $\hat{\alpha} = 0.1$ and end at $k = 72 <$ max_iter=100

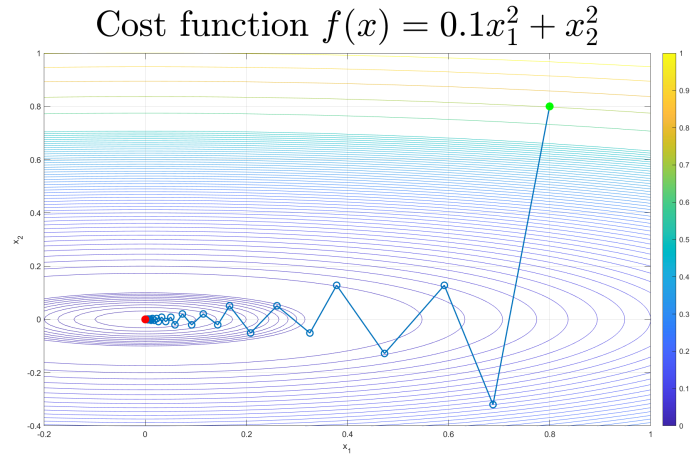
Figure: Left: Newton Raphson. Right: Steepest descent

Steepest descent vs Newton Raphson

Compareson between steepest descent and Newton Raphson when using backtracking line search



(a) $\hat{\alpha} = 0.1$, $\beta = 0.7$ and end at $k = 2 <$ max_iter=100



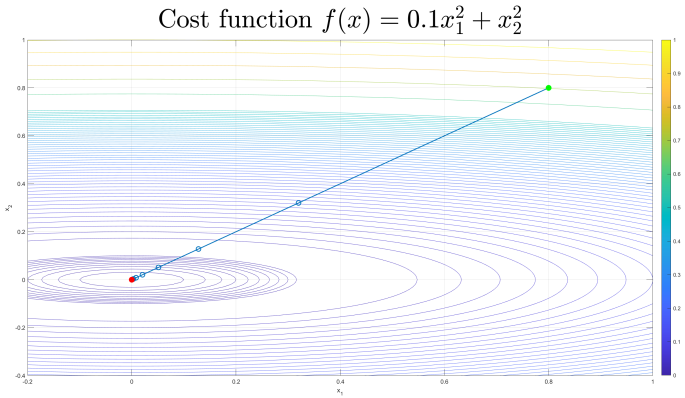
(b) $\hat{\alpha} = 0.1$. $\beta = 0.7$ and end at $k = 63 <$ max_iter=100

Figure: Left: Newton Raphson. Right: Steepest descent

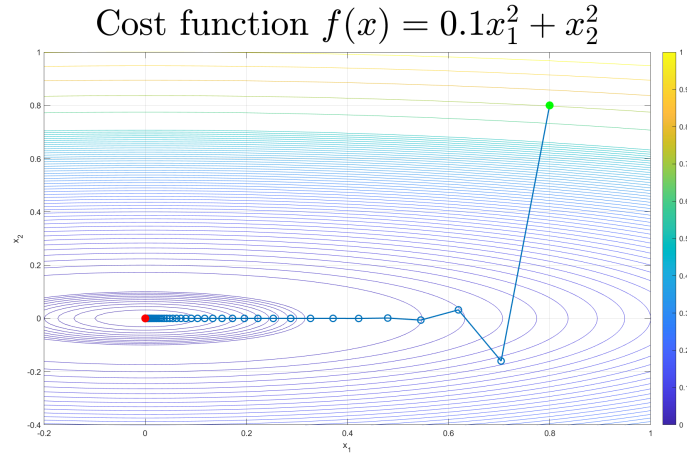
Steepest descent vs Newton Raphson

Compareson between steepest descent and Newton Raphson when using

$\alpha_k = \hat{\alpha}$, with $\hat{\alpha}$ a fixed tuning constant, sometimes called a learning rate.



(a) $\hat{\alpha} = 0.6$ and end at $k = 16 <$ max_iter=100



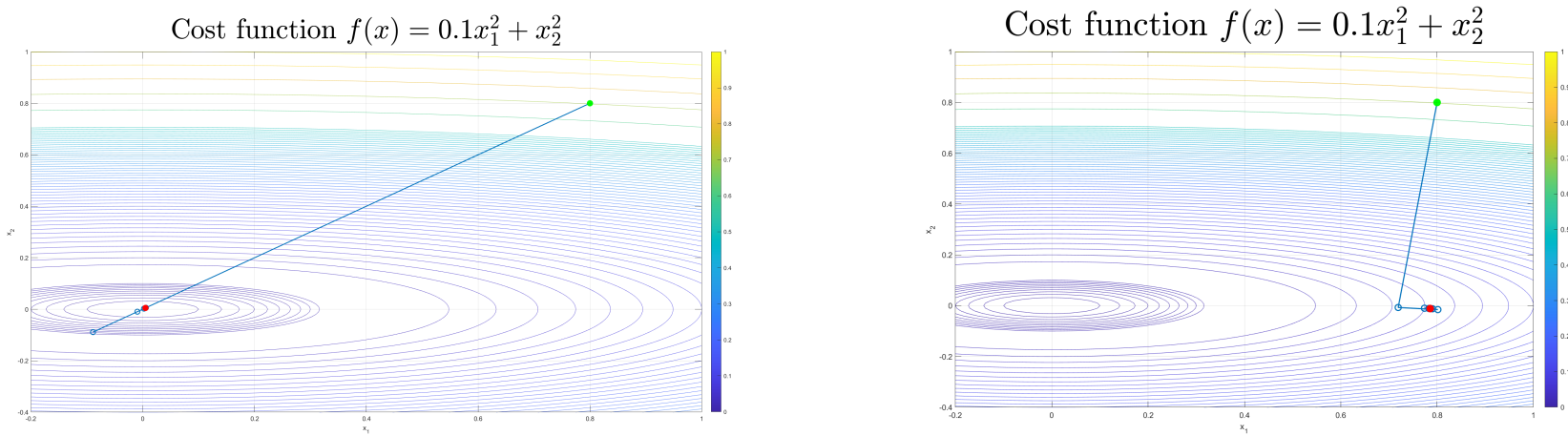
(b) $\hat{\alpha} = 0.6$ and end at $k = 91 <$ max_iter=100 (**FAIL** for $\hat{\alpha} = 1$ and reach max_ither for $\hat{\alpha} < 0.55$)

Figure: Left: Newton Raphson. Right: Steepest descent

Steepest descent vs Newton Raphson

Compareson between steepest descent and Newton Raphson when using

$$\alpha_k \approx \frac{\hat{\alpha}^2 ||d_k||_2^2}{2(f(x_k+\hat{\alpha}d_k)-f(x_k)+\hat{\alpha}||d_k||_2^2)}, \text{ with } \hat{\alpha} = \alpha_{k-1},\ k > 1 \text{ (the previous optimum) and } \hat{\alpha} = 1 \text{ for } k = 1.$$



(a) End at $k = 16 <$ max_iter=100

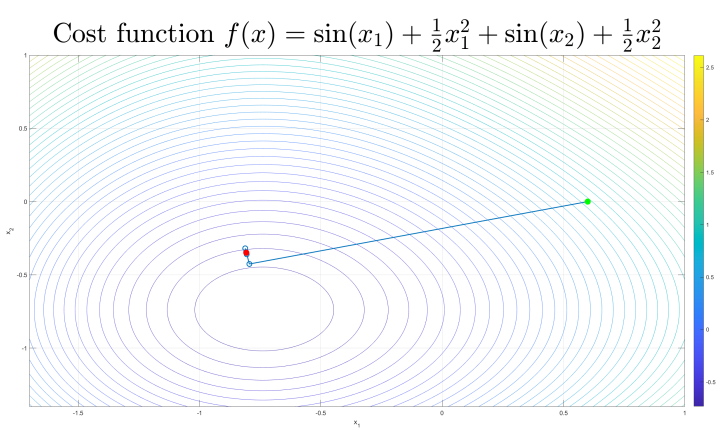
(b) End at $k = 18 <$ max_iter=100 **FAILED**

Figure: Left: Newton Raphson. Right: Steepest descent

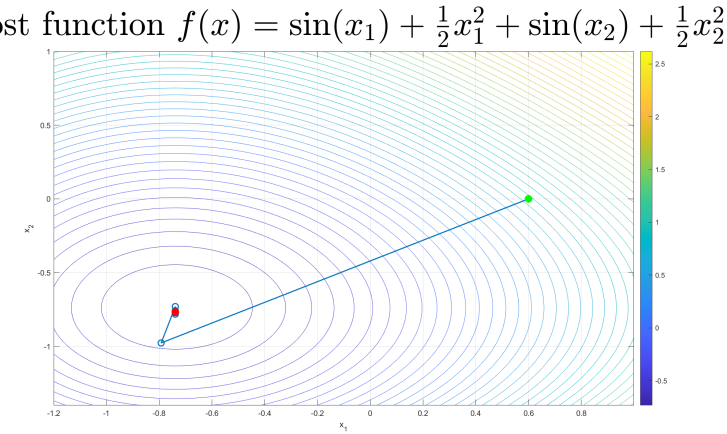
Steepest descent vs Newton Raphson

Compareson between steepest descent and Newton Raphson when using

$$\alpha_k \approx \frac{\hat{\alpha}^2 ||d_k||_2^2}{2(f(x_k+\hat{\alpha}d_k)-f(x_k)+\hat{\alpha}||d_k||_2^2)}, \text{ with } \hat{\alpha} = \alpha_{k-1},\ k > 1 \text{ (the previous optimum) and } \hat{\alpha} = 1 \text{ for } k = 1.$$



(a) End at $k = 14 <$ max_iter=100 **FAILED**



(b) End at $k = 19 <$ max_iter=100

Figure: Left: Newton Raphson. Right: Steepest descent

Gauss-Newton method

The next optimization method we will describe deals with optimization problems of vector-valued functions and is called the Gauss-Newton method.

More precisely, for a given function

$$F : \mathbb{R}^n \to \mathbb{R}^m; x \mapsto F(x) = [f_1(x) \cdots f_m(x)]^\top,$$

we wish to find a point x^* such that $F(x) = 0$ (i.e., $f_i(x^*) = 0$ for all i).

In order to solve this problem we define the real-valued function

$$f : \mathbb{R}^n \to \mathbb{R}; x \mapsto f(x) = F(x)^T F(x) = ||F(x)||_2^2 = \sum f_i(x)^2.$$

It follows that solving the (uncontrained) optimization problem

$$\min_{x \in \mathbb{R}^n} f(x)$$

is equivalent to finding the zeros of F.

Note that we are trying to minimize a sum of squares, that is, we are trying to solve a least squares problem.

Gauss-Newton method

A point x^* is a zero of

$$F : \mathbb{R}^n \to \mathbb{R}^m; x \mapsto F(x) = [f_1(x) \cdots, f_m(x)]^\top,$$

that is, $F(x^*) = 0$

iff

$$x^* \in \text{argmin}_{x \in \mathbb{R}^n} f(x)$$

with

$$f : \mathbb{R}^n \to \mathbb{R}; x \mapsto f(x) = F(x)^T F(x) = \|F(x)\|_2^2 = \sum f_i(x)^2.$$

Hence, x^* may be obtained by means of the Newton method

$$x_{k+1} = x_k + \alpha_k d_k, \quad d_k = -H_f(x_k)^{-1} \nabla f(x_k),$$

with

$$\nabla f(x) = 2 J_F(x)^\top F(x)$$
$$H_f(x) \approx 2 J_F(x)^\top J_F(x).$$

Stochastic gradient descent

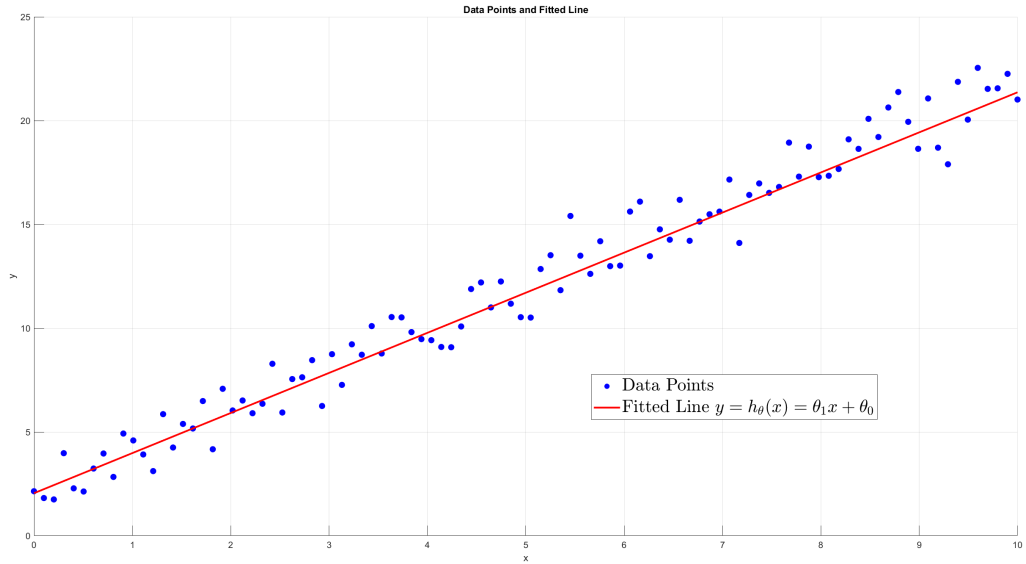
The gradient descent method can also be used for parameter estimation.

Let $\{(x_i, y_i)\}_{i=1}^N$ be a set of N data points and $y = h_\theta(x)$ be a function with θ a vector of unknown parameters. Assume that we want to choose θ such that $y = h_\theta(x)$ fits the data set as best as possible. If we interpret 'as best as possible' by means of the 2-norm, this can be cast as the optimization problem:

$$\min_\theta \sum_{i=1}^N \|y_i - h_\theta(x_i)\|_2^2$$

Hence the cost function is

$$f(\theta) = \sum_{i=1}^N \|y_i - h_\theta(x_i)\|_2^2$$



Stochastic gradient descent

In many applications, particularly in machine learning and deep learning, the data set can be large (i.e., N is large). This can potential make it computationally hard to solve the optimization problem

$$\min_{\theta} \sum_{i=1}^{N} \|y_i - h_\theta(x_i)\|_2^2 \tag{1}$$

In such situations it is often an advantage to update the model parameters θ using small random subsets of the data set called mini-batches. More detailed, instead of solving the optimization problem (1), we partition the data set into, say p, mini-batchs of size N_j with $N_1 + \cdots N_p \geq N$, and then recursively ($j = 1, \ldots, p$) solve

$$\min_{\theta} \sum_{i=1}^{N_j} \|y_i - h_\theta(x_i)\|_2^2$$

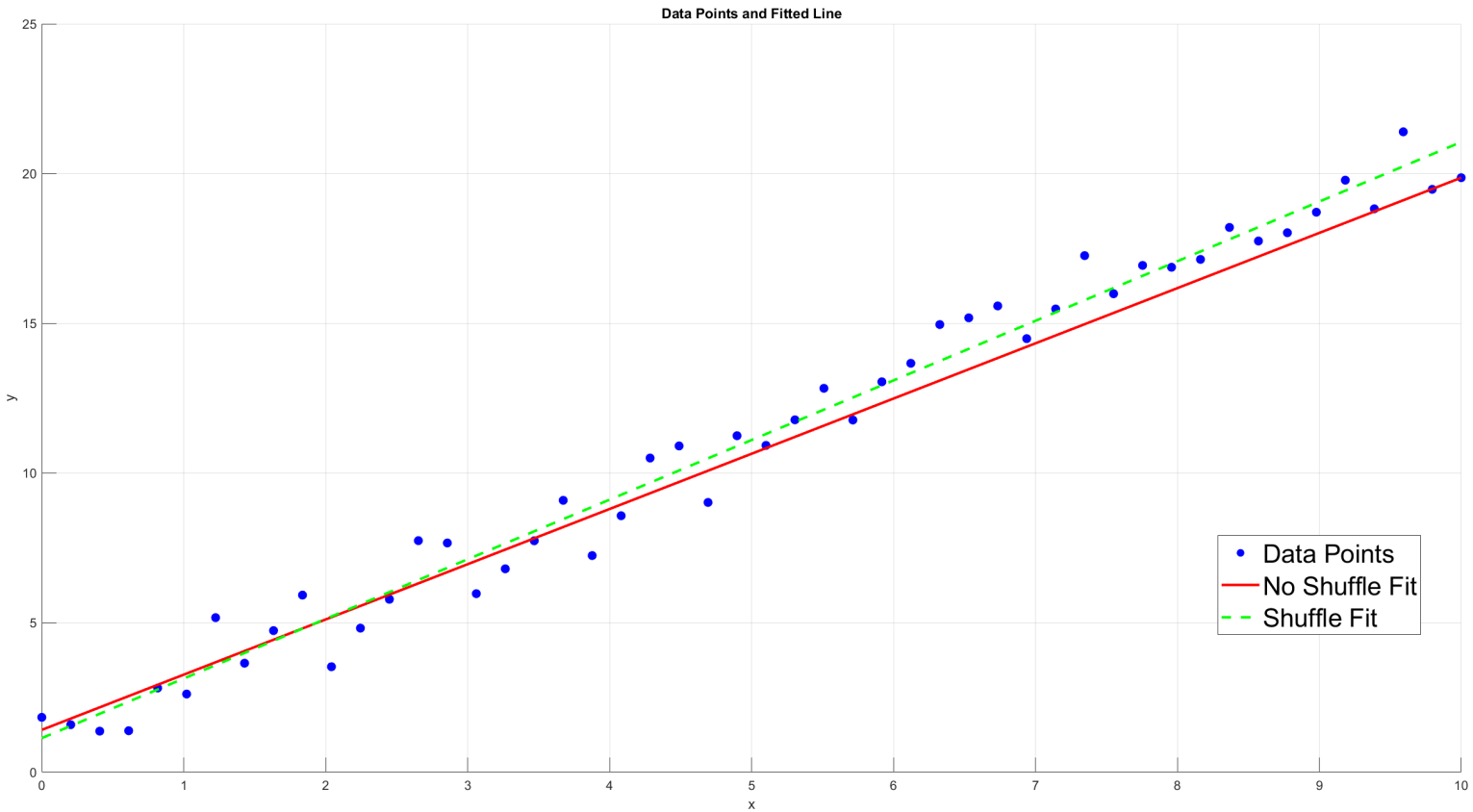
using the jth solution as initial condition for the $(j+1)$th iteration, with an initial guess for $j = 1$. Moreover, when the data is ordered or structured in some way, each mini-batch should be randomly shuffled in order to avoid order bias.

When the steepest (or gradient) descent method is combined with mini-batching and/or shuffling, it is referred to as stochastic gradient descent.

Example

Data generated by $y_i = \theta_1 x_i + \theta_0 + w_i = 2x_i + 1 + w_i$ with $w_i \sim N(0, 1)$ a (discrete-time) gaussian stochastic process with mean 0 and variance 1.

- No shuffling: Minimizer is $[\theta_0\ \theta_1]^\top = [1.4138\ 1.8446]^\top$ with (2-norm) error 0.4420
- Shuffling: Minimizer is $[\theta_0\ \theta_1]^\top = [1.1383\ 1.9914]^\top$ with (2-norm) error 0.1385



Example

Data generated by $y_i = \theta_1 x_i + \theta_0 + w_i = 2x_i + 1 + w_i$ with $w_i \sim N(0, 1)$ a (discrete-time) gaussian stochastic process with mean 0 and variance 1.

- No shuffling: Minimizer is $[\theta_0\ \theta_1]^\top = [1.4138\ 1.8446]^\top$ with (2-norm) error 0.4420
- Shuffling: Minimizer is $[\theta_0\ \theta_1]^\top = [1.1383\ 1.9914]^\top$ with (2-norm) error 0.1385